



Robotics & Computer Vision

Week 9: Mapping from AI to Grasping

Hand-Eye Calibration • Camera $\rightarrow$ Robot Base • Generating T_{target} for IK

Lê Đình Phong, Ph.D

1

Understand the Full Pipeline

Camera image → AI detection → 6D pose in camera frame → mapping to base → IK → robot grasps object. See every step clearly connected.

2

Master Hand-Eye Calibration

Grasp the concept and core equation of hand-eye calibration: $AX = XB$. Understand what it solves and why it is indispensable.

3

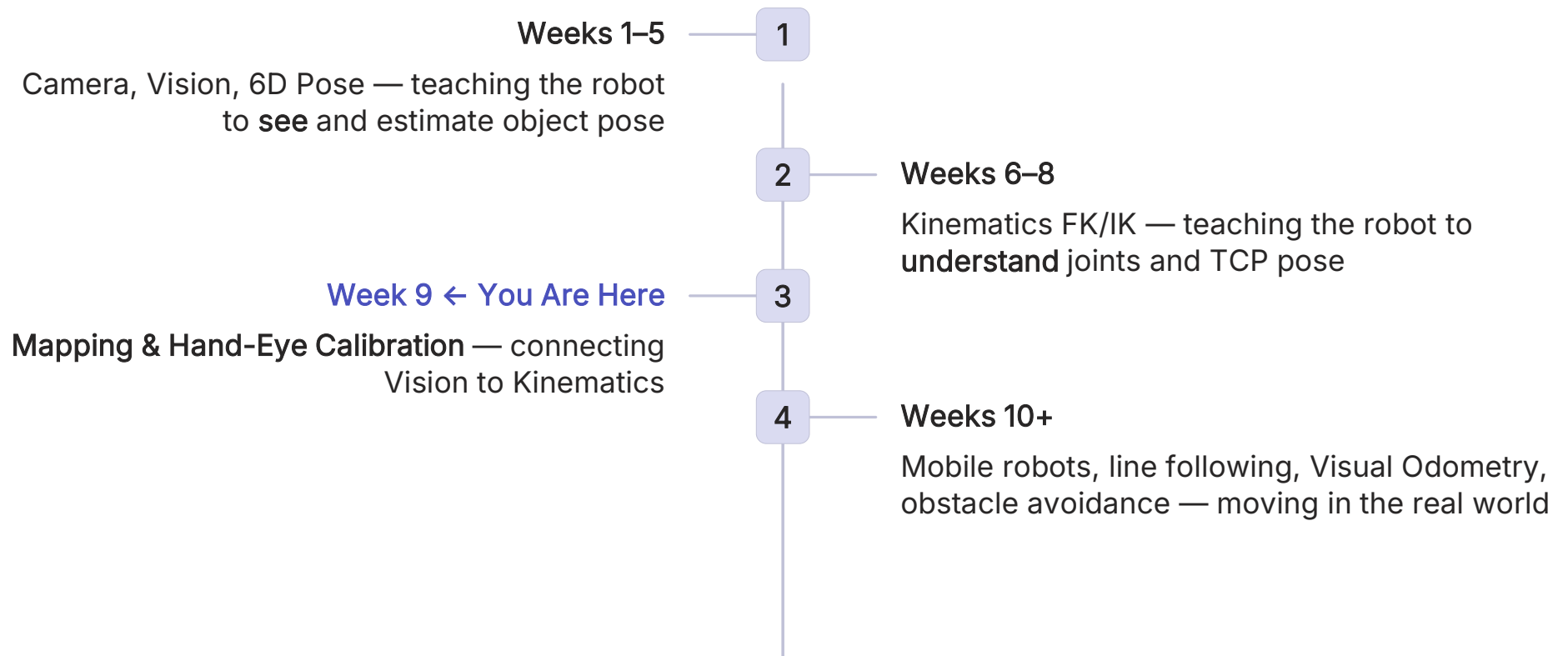
Implement the Mapping Code

Practice writing transformations: pixel → 3D ray in camera frame → 3D point in camera → coordinates in base frame.

Where Does Week 9 Fit?

Week 9 — Mapping from AI to Grasping

The course is structured as three progressive pillars that culminate in a fully integrated AI-to-Robot demo. Week 9 is the critical bridge between perception and motion.



The Four Essential Frames

Frame C — Camera

Origin at the camera's optical center. Z-axis points forward (into the scene). All AI pose outputs are initially expressed here.

Frame B — Base

Origin at the robot's base or `base_link`. The reference frame for IK and motion planning. All grasping targets must be expressed here.

Frame T — Tool

Located at the TCP (Tool Center Point). Defines where the gripper makes contact. Offset from the object frame is defined here.

Frame O — Object

Centered on the object to be grasped. Estimated by the 6D pose network. Relates camera measurements to the real-world object geometry.

The fundamental transform chain

Week 9 — Mapping from AI to Grasping

Goal

We need T_{target} — the TCP pose expressed in the base frame B — to feed directly into the IK solver from Week 8.

The Chain

p_C (from vision AI)

↓ × T_{BC}

p_B (in robot base)

↓ × T_{offset}

T_{target} (TCP goal for IK)

Where T_{BC} Comes From

T_{BC} is the result of hand-eye calibration — it encodes the fixed spatial relationship between the camera and the robot base.

Why This Matters

Without an accurate T_{BC} , every single grasp will be offset by the same systematic error, regardless of how good the AI pose network is.

Intrinsics — Matrix K

Encodes the camera's internal optics: focal length (f_x , f_y), principal point (c_x , c_y), and lens distortion coefficients. **Constant as long as the lens is unchanged.** Determined once during camera calibration (e.g., checkerboard method from Weeks 2–3).

Extrinsics — R , t

Describe the camera's position and orientation relative to another frame (e.g., the robot base). Extrinsics **change** whenever the camera is physically moved or remounted. This is exactly what hand-eye calibration solves.

Full Mapping Requires Both

Intrinsics map 3D camera-frame points to 2D pixels. Extrinsics map camera-frame points to base-frame points. You need **both** for a complete pixel $\rightarrow$ base transform.

The Pinhole Camera Equation

Week 9 — Mapping from AI to Grasping

In homogeneous coordinates, the full projection from 3D world point to 2D pixel is written as a compact matrix product:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[R \mid t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Variables

- (X, Y, Z) — 3D point in world or camera frame
- (u, v) — pixel coordinates in the image
- s — projective scale factor (depth)

Matrices

- K — 3×3 intrinsics matrix
- $[R \mid t]$ — 3×4 extrinsics matrix
- $K[R \mid t]$ — full 3×4 projection matrix P

 This equation is the mathematical backbone of all camera-to-world mapping in this course.

From Pixel (u, v) to a Ray in Camera Frame

Week 9 — Mapping from AI to Grasping

Inverting the intrinsics matrix K gives us the **direction** of the viewing ray through the pixel — but not yet the depth.

$$\begin{bmatrix} x_c \\ y_c \\ 1 \end{bmatrix} = K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

What We Get

A unit direction vector in frame C pointing toward the scene. This tells us *where* the object is in angular terms, but we still need depth Z to recover the full 3D location.

Where Depth Comes From

- 6D pose network output (direct Z)
- Stereo camera disparity
- Time-of-Flight (ToF) / depth sensor
- Known planar constraint ($Z = Z_o$)

Recovering the 3D Point in Camera Frame

Week 9 — Mapping from AI to Grasping

Once depth Z is known (from the 6D pose network or depth sensor), computing the full 3D camera-frame coordinates is straightforward:

$$X_C = Z \cdot x_c, \quad Y_C = Z \cdot y_c, \quad Z_C = Z$$

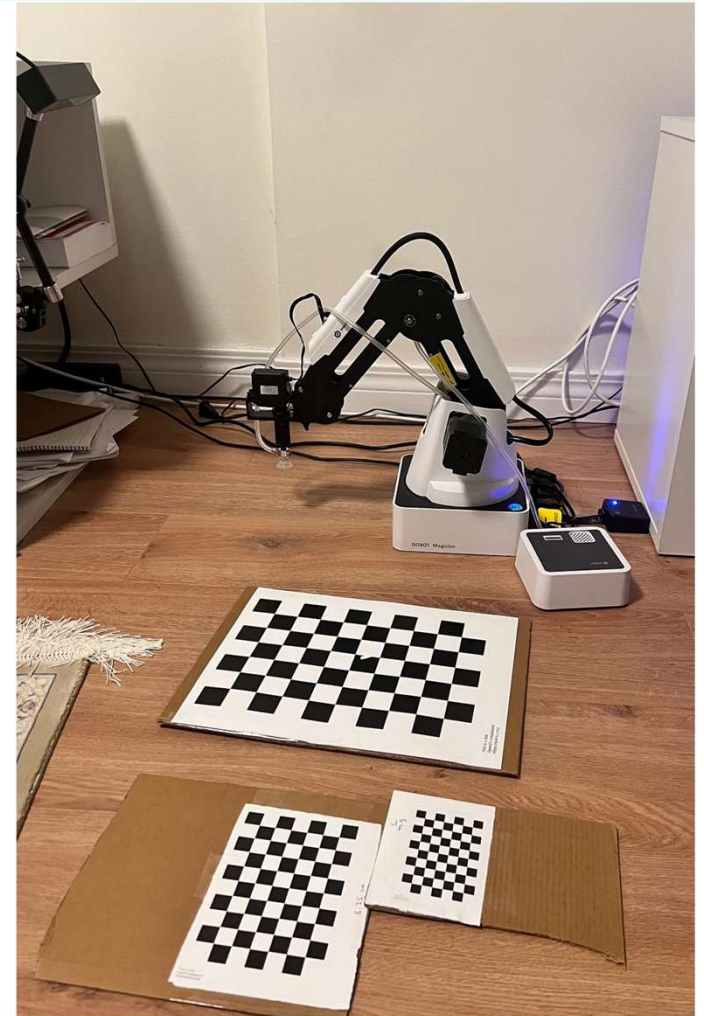
where $[x_c, y_c, 1]^T = K^{-1}[u, v, 1]^T$ from the previous step.

- ✔ In practice, the 6D pose network directly outputs (X_C, Y_C, Z_C) as part of the estimated pose T_{CO} — no manual depth reconstruction needed in the standard pipeline.

This 3D point in frame C is the starting input for all subsequent mapping transformations.

Intuition: What Is Hand-Eye Calibration?

Find the fixed transform T_{BC} between the camera (eye) and the robot base (hand's reference) by observing a calibration target from multiple robot poses.



The $AX = XB$ Equation

Week 9 — Mapping from AI to Grasping

Notation

A_i = motion of end-effector between pose i and $i+1$

(measured from joint encoders via FK)

B_i = motion of camera observing calibration target between the same two poses

(measured from image via PnP)

X = T_{EC} or T_{CE} — the unknown camera-to-EE transform

The Equation

$$A_i X = X B_i$$

By collecting $N \geq 3$ non-collinear robot motions and their corresponding camera observations, we can solve for X using least-squares or closed-form methods.

Intuition

The left side tracks how the *robot* moved; the right side tracks how the *camera* saw the target move. X is the only fixed transform that makes them consistent.

Solving $AX = XB$ — Overview

01

Collect Motion Pairs

Move the robot to N different poses ($N \geq 3$, ideally 10–20). Record each (A_i, B_i) pair. More diverse motions $\rightarrow$ better conditioning.

03

Solve with a Library

In this course, students use existing implementations (OpenCV `calibrateHandEye`, or Python `spatialmath` utilities). Custom derivation is not required.

02

Decouple Rotation & Translation

Most solvers first extract the rotation part of X (using quaternion or dual-quaternion algebra), then solve the translation linearly given the known rotation.

04

Validate the Result

Place a known marker at a measured base-frame position. Map it through T_{BC} and compare with ground truth to quantify calibration error.

Using the Calibration Output

After calibration, we have T_{BC} — the pose of camera frame C expressed in base frame B. This single matrix is the cornerstone of the entire mapping pipeline.

$$p_B = T_{BC} p_C$$

Every Point, Every Time

Any 3D point measured in camera coordinates (from AI, stereo, or depth sensor) can be instantly converted to base coordinates using this single matrix multiplication. The transform is fixed as long as the camera mount does not move.

Inverse: T_{CB}

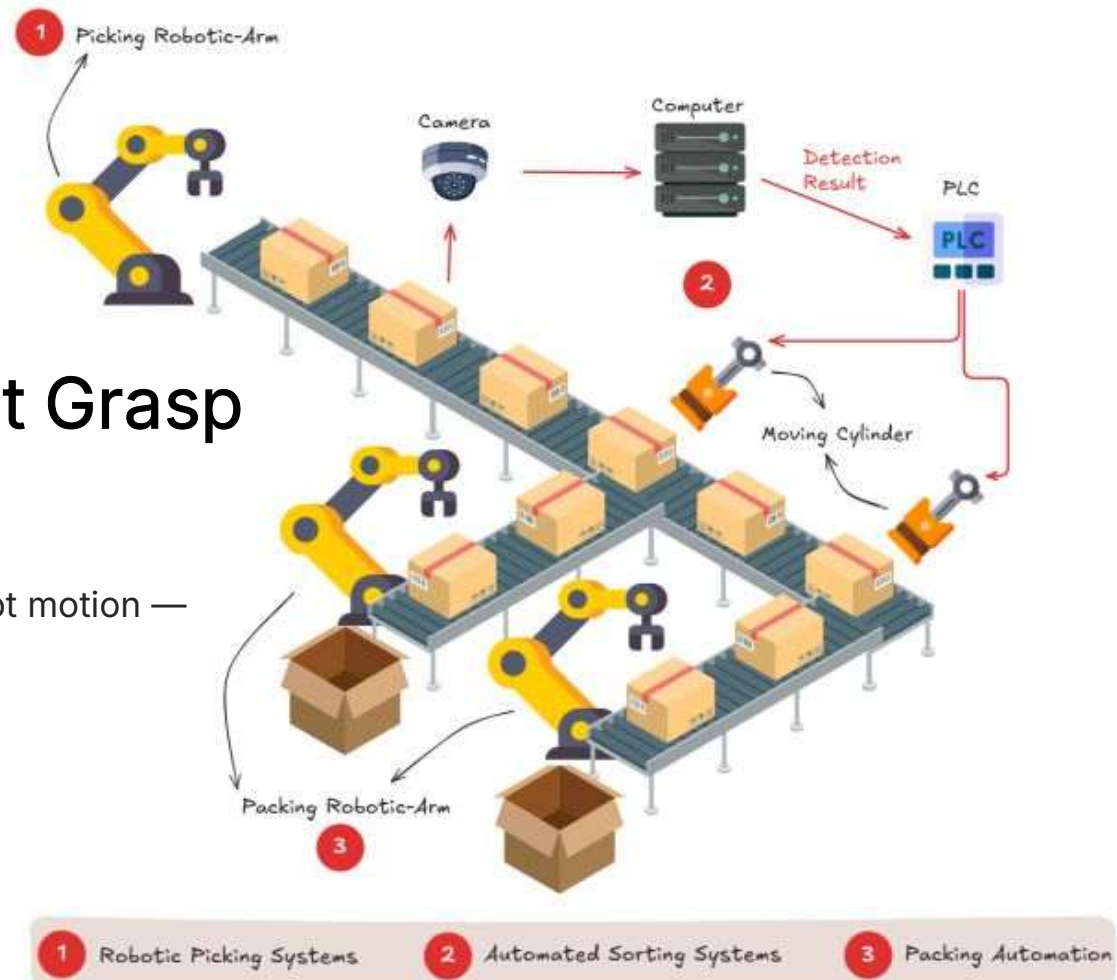
If you have T_{BC} and need the reverse direction:

```
T_CB = np.linalg.inv(T_BC.A)
```

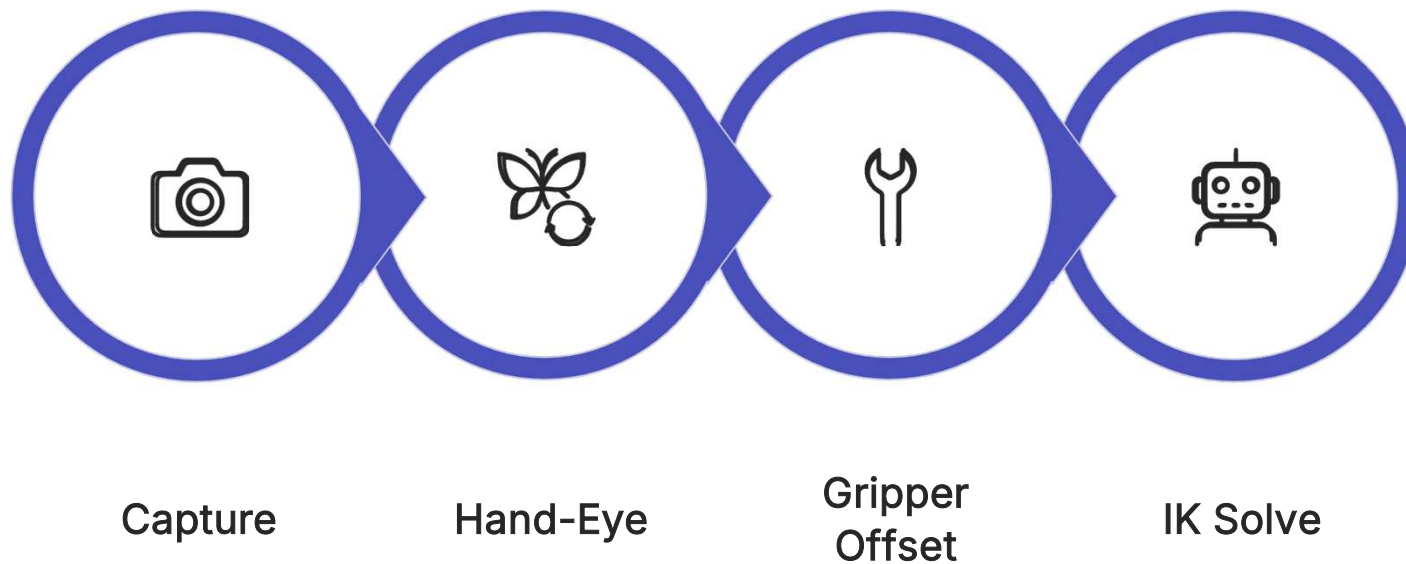
or equivalently `T_BC.inv()` in `spatialmath`. Be careful — mixing T_{BC} and T_{CB} is the most common student mistake.

The Complete AI → Robot Grasp Pipeline

Four clearly-defined steps from raw image to robot motion — each building on the last.



Pipeline: Step by Step



This four-step pipeline is the central deliverable of Week 9 and the backbone of the end-of-course AI → Robot demo.

All Transforms in the Grasping Problem



T_{BC}

Pose of camera in base frame. Result of hand-eye calibration. **Fixed** as long as camera mount is unchanged.



T_{CO}

Pose of object in camera frame. Output of the 6D pose network from Week 5. Changes with every new image capture.



T_{OT}

Offset of TCP relative to the object. Defined by the gripper design and grasp strategy. Set once during system design.



T_{BT} (= T_{target})

TCP pose in base frame — the final IK input.
Computed as: $T_{BO} \times T_{OT}$, where $T_{BO} = T_{BC} \times T_{CO}$.

Computing T_{target} for IK

The chain of matrix multiplications that transforms the AI output into an actionable robot command:

$$T_{BO} = T_{BC} \ T_{CO}$$

$$T_{BT} = T_{BO} \ T_{OT}$$

$$T_{\text{target}} = T_{BT}$$

- ✓ T_{target} is the direct input to the 6-DOF IK solver built in Week 8. The pipeline is now complete — from camera observation to joint angles.

Note the order of multiplication: **right-to-left composition**. T_{BC} acts on the result of T_{CO} — always verify your frame chain before coding.

Choosing the Grasp Offset T_{OT}

What Is T_{OT} ?

T_{OT} encodes *how* the gripper approaches the object. It depends on the tool type, gripper geometry, and approach strategy. It is a design decision — not estimated from data.

Common Grasp Strategies

- Top-down grasp: TCP above object, offset along z-axis
- Side grasp: TCP offset along x or y
- Vacuum suction: approach normal to surface

Example: Top-Down Grasp

TCP 5 cm above the object center, aligned with object y

$$T_{OT} = \text{Trans}(\theta, \theta, d) * R_z(\text{yaw_offset})$$

where $d = 0.05$ meters (safe standoff distance perpendicular to table).

Rule of Thumb

Design grippers to be "**forgiving**": wide contact surfaces, vacuum suction, or chamfered guides compensate for small pose estimation errors.

Numerical Example: Object Pose $\rightarrow$ T_{target}

```
from spatialmath import SE3

# Hand-eye calibration result: camera 20cm in front, 30cm above base
T_BC = SE3(0.2, 0.0, 0.3)

# 6D pose network output: object 15cm in front, 5cm to the side
T_CO = SE3(0.15, 0.05, 0.0)

# Gripper offset: TCP 5cm above object along object z-axis
T_OT = SE3(0, 0, 0.05)

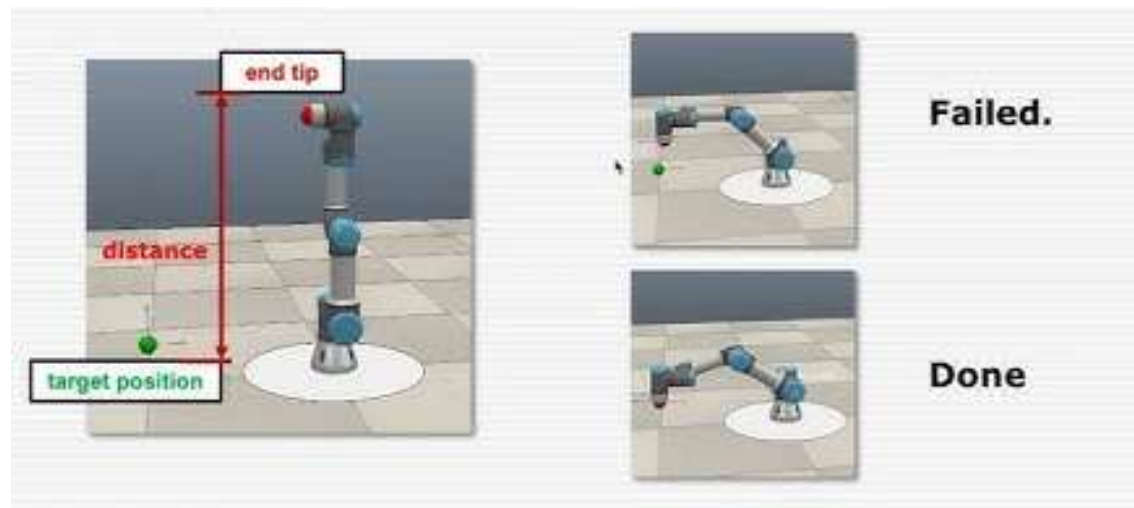
# Chain the transforms
T_BO = T_BC * T_CO
T_BT = T_BO * T_OT

print(T_BT.A) # 4x4 homogeneous matrix – feed to IK
```

 Use this example in the lab to verify your intuition about mapping before connecting to the real robot or Swift simulator.

Week 9 Lab — Overview

Eight progressive lab exercises guide students from raw pixel inputs through to a complete simulated grasp pipeline, culminating in hardware demo on Swift or Yahboom arm.



Lab 9.1 — Pixel to Viewing Ray

Build the foundation function that converts a 2D pixel into a 3D direction vector in camera frame C.

```
def pixel_to_ray(K, u, v):  
    uv1 = np.array([u, v, 1.0])  
    x_c = np.linalg.inv(K) @ uv1  
    dir_c = x_c / np.linalg.norm(x_c)  
    return dir_c
```

Input

- K — 3×3 intrinsics matrix (from Weeks 2–3 calibration)
- (u, v) — pixel coordinates of detected object center

Output

- Unit direction vector in frame C
- Points toward the object but carries no depth information yet

Lab 9.2 — 3D Point in Camera Frame

Given the 6D pose output from the AI network, construct a proper pose struct and verify consistency with the 2D detection.

Task

Build a `PoseC` struct holding rotation `R_C_0` and translation `t_C_0` — the full pose of the object in camera frame.

Verification

Project the 3D pose back to 2D using `K` and compare with the bounding box center from the detector. Error should be < 2 pixels.

Key Insight

This round-trip test (3D $\rightarrow$ 2D $\rightarrow$ compare with detector) is one of the most powerful sanity checks in the pipeline.

Lab 9.3 — Mapping Point $p_C \rightarrow p_B$

Implement the core mapping function using a pre-loaded calibration T_{BC} from a config file.

```
def cam_to_base(T_BC, p_C):  
    pC_h = np.r_[p_C, 1.0]      # append homogeneous coordinate  
    pB_h = T_BC.A @ pC_h        # 4x4 matrix × 4x1 vector  
    return pB_h[:3]            # extract XYZ
```

⚠ Always append the homogeneous 1 before multiplying. A common bug is forgetting this and getting a 3×3 rotation-only result instead of the full rigid transform.

Test with several known points and **visualize** the transformed cloud in a 3D scatter plot to confirm axes and directions are correct.

Lab 9.4 — Mapping Full Pose $T_{CO} \rightarrow T_{BO}$

```
def object_pose_in_base(T_BC, T_CO):  
    return T_BC * T_CO # SE3 matrix multiplication
```

Exercise

Print the resulting R and t components of T_{BO} . Interpret them geometrically: where is the object in the base frame? Is it in front of, behind, or to the side of the robot?

Expected Insight

Students should be able to **read** a 4×4 SE3 matrix and immediately locate the object in 3D space. This geometric intuition is critical for debugging real-world deployments.

Lab 9.5 — Build T_{target} and Run IK

```
def build_target(T_B0, T_OT):  
    T_BT = T_B0 * T_OT  
    return T_BT  
  
# Use IK from Week 8  
T_target = build_target(T_B0, T_OT)  
q_target = ik_solver(robot, T_target)  
print("Joint angles:", q_target)
```

- ✓ This is the **first time** in the course that the full chain — AI pose → mapping → IK → joint angles — runs end-to-end in student code. Celebrate the milestone!

Lab 9.6 — Full Pipeline with Simulated AI Data

Scale up: mock a scene with multiple objects and run the complete pipeline for each.

```
# Simulated AI output: 3 objects on the table
objects = [SE3(0.15, 0.05, 0.0), # Object 1
           SE3(0.20, -0.03, 0.0), # Object 2
           SE3(0.10, 0.08, 0.0)] # Object 3

for T_CO in objects:
    T_BO = T_BC * T_CO
    T_BT = T_BO * T_OT
    q = ik_solver(robot, T_BT)
    print(f"Grasp joint angles: {q}")
```

This simulates the robot sequentially grasping all detected objects — a preview of the final course demo scenario.

Lab 9.7 — Effect of Calibration Errors

Deliberately corrupt T_{BC} with small errors to quantify their effect on grasp accuracy.

Translation Error

Add 1–2 cm offset to the t component of T_{BC} . Observe how the grasp point shifts in a fixed direction — a classic sign of calibration drift.

Rotation Error

Apply a small rotation ($2\text{--}3^\circ$) to the R component. Observe how grasp error grows with object distance from the camera optical axis.

Key Takeaway

Even small calibration errors cause **systematic** (not random) grasp failures — the robot always misses in the same direction. This is the diagnostic signature of a calibration problem.

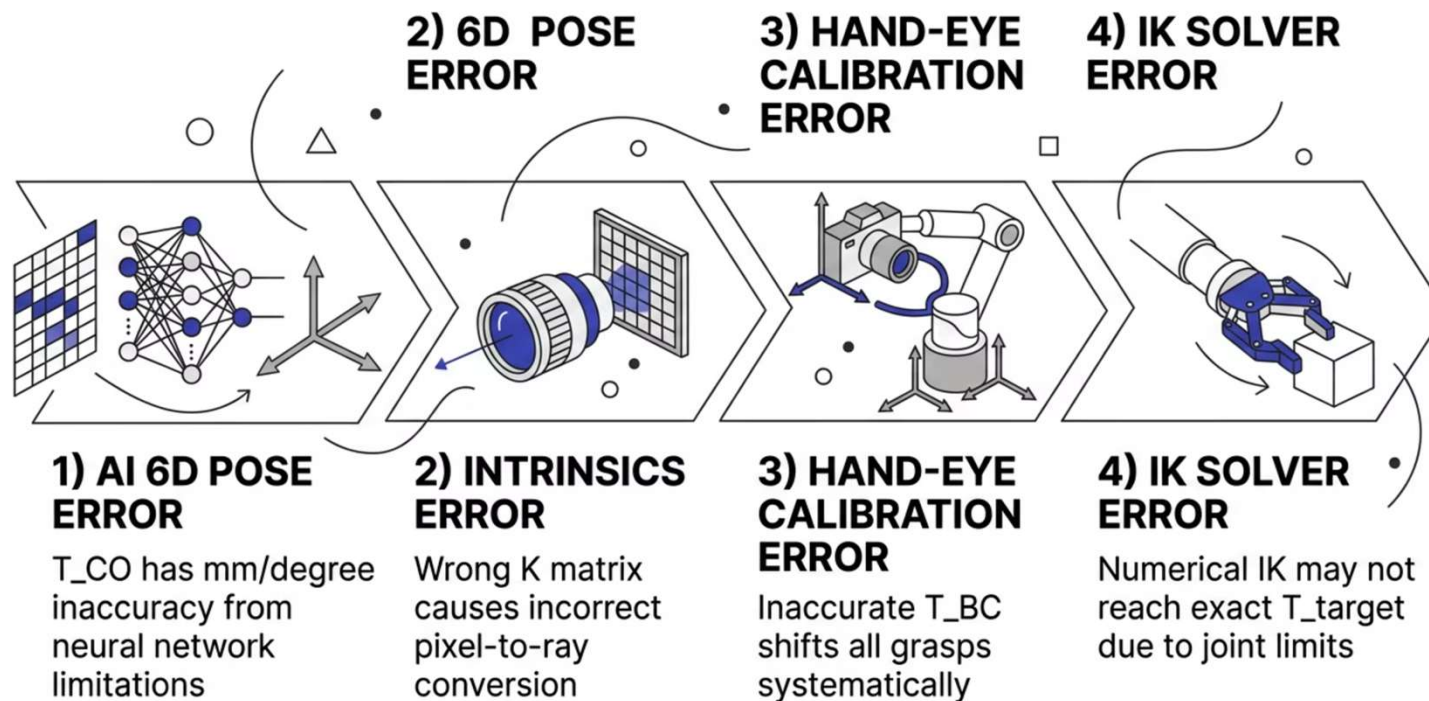


LAB 9.8

Lab 9.8 — Connect to Real Hardware

Apply the full pipeline to a physical robot. Place 1–2 objects at known positions on the table. Run mapping + IK and command the arm to approach each grasp pose. If physical grasp is not yet configured, visualize the trajectory in Swift simulator and confirm correctness visually.

Sources of Error in the Grasp Pipeline



Errors compound through the chain. The cumulative effect can easily exceed the tolerance of a standard gripper. Design the gripper to compensate for the expected total error budget.

Impact of AI Pose Errors

What Happens

If T_{CO} from the 6D pose network contains errors of a few mm or a few degrees, T_{target} will be offset by the same amount. This error is then fed directly into IK, which solves correctly — but for the *wrong* target position.

Mitigation Strategies

- Use larger, more textured objects for better pose estimation
- Average pose estimates over multiple frames
- Design grippers with wide contact surfaces
- Use vacuum suction or chamfered guides for tolerance

 The IK solver always "succeeds" even when the target is wrong. All quality control must happen upstream — at the camera and calibration stage.

Impact of Hand-Eye Calibration Errors

Systematic Offset

A wrong T_{BC} shifts *every* grasp in the same direction by the same amount. If the robot consistently misses objects to the left by 3 cm, suspect a calibration error — not a pose estimation or IK issue.

When to Recalibrate

Recalibrate hand-eye whenever: the camera mount is accidentally bumped, the camera bracket is reassembled, or you observe a new systematic grasp offset that wasn't present before.

Best Practice

In production systems, build a **periodic recalibration routine**. Run it at the start of each work session or after any physical intervention on the robot. Takes less than 5 minutes with a proper calibration fixture.

Mapping, IK, and Their Tight Coupling

The Chain of Responsibility

Mapping produces T_{target} . IK converts T_{target} into joint angles q_{target} . The IK solver is blind to whether T_{target} is correct — it simply finds the best solution to reach the given pose.

Consequence: Debugging a missed grasp requires tracing backward through the chain to find where the error was introduced.

Debugging Order

1 Check calibration first

Is T_{BC} accurate? Use ground-truth markers.

2 Check AI pose

Is T_{CO} reasonable? Project back to pixels.

3 Check IK

Did IK converge? Check residual error.

Extending to Mobile Robots

The same mapping concept applies to mobile robots, with one additional frame: the world/map frame W.

```
# T_WB from SLAM or odometry (pose of base in world/map)
# T_BC from hand-eye calibration (fixed)

p_C = np.array([Xc, Yc, Zc, 1.0]) # point in camera frame

p_B = T_BC.A @ p_C                # camera → base
p_W = T_WB.A @ p_B                # base → world/map
```

❗ The path planner uses p_W to decide where to navigate. The structure is identical to the arm mapping — just one extra frame in the chain.

Homography: A Simpler Alternative for Flat Scenes

When to Use Homography

If objects lie on a known flat surface ($Z = Z_0$), a homography matrix H directly maps pixel coordinates to 2D table coordinates — no 6D pose needed. Useful for pick-and-place on flat conveyor belts or tabletops.

6D Pose vs. Homography

- **6D Pose:** full $(x,y,z,roll,pitch,yaw)$ — flexible, general, requires more data
- **Homography:** 2.5D only — simpler, but limited to flat objects and fixed camera

RCV prioritizes **6D pose** to develop general concepts applicable to all future scenarios.

Connecting Weeks 5 → 9: The Full Thread

1

Week 5

AI 6D pose network outputs T_{CO} — object pose in camera frame

2

Weeks 6–7

Product of Exponentials (PoE) and FK: $F(q) = T$ — forward kinematics

3

Week 8

Inverse Kinematics: $T_{target} \rightarrow q_{target}$ — joint angles from desired pose

4

Week 9

Mapping: $T_{CO} \rightarrow T_{target}$ — the bridge between camera and robot

Each week adds one essential piece. Week 9 is the glue that makes the entire system functional as a coherent whole.

Common Mapping Errors to Avoid



Wrong Multiplication Order

Writing $T_{CO} * T_{BC}$ instead of $T_{BC} * T_{CO}$. Matrix multiplication is **not commutative**. Always trace the frame chain: $B \leftarrow C \leftarrow O$.



Using T_{CB} Instead of T_{BC}

If your calibration output is T_{CB} (camera to base, reversed), use its inverse: $T_{BC} = T_{CB}.inv()$. Confusing the two is the second most common bug.



Unit Mismatch

Mixing meters and centimeters in the same transform chain. Always normalize to **meters** throughout. Pixel coordinates should never appear in the 3D transform chain.

How to Debug Camera → Base Mapping

01

Place a Known Marker

Put a calibration marker (e.g., ArUco tag or checkerboard corner) at a precisely measured position in the base frame. Record its ground-truth coordinates.

02

Detect and Map

Detect the marker in the camera image, get its 3D camera-frame coordinates p_C , apply T_{BC} to get $p_{B_estimated}$.

03

Compute Error

Compare $p_{B_estimated}$ with ground-truth. Measure the error vector $(\Delta x, \Delta y, \Delta z)$.

04

Diagnose from Error Pattern

Consistent offset in one axis → wrong T_{BC} translation. Rotating error with distance → wrong R component. Large error everywhere → possible intrinsics problem.

Suggested Lab Structure for Week 9

Part 1 — Simulation

All 8 labs run in Python using `spatialmath` and `roboticstoolbox`. No hardware required. Students build intuition safely and quickly iterate on bugs.

Part 2 — Calibration Demo

Instructor demonstrates a pre-run hand-eye calibration via video or offline script. Students examine the output `T_BC` and run Labs 9.3–9.5 with it.

Part 3 — Hardware Demo

If equipment is available: connect to Swift simulator or real Yahboom arm. Run the full pipeline for 1–2 simple tabletop objects. Celebrate the first complete AI → Robot grasp!

What Students Can Do After Week 9

Write the Mapping Function

Implement $\text{pixel} \rightarrow \text{ray} \rightarrow 3\text{D point in C} \rightarrow \text{transform to B using } T_{BC}$ correctly.

Interpret Each Transform

Explain the role and origin of T_{BC} , T_{CO} , T_{OT} , and T_{BT} — geometrically and mathematically.

Build the Complete Pipeline

Chain mapping + IK into working code that takes simulated AI output and produces robot joint angles.



Preparing for Weeks 10–12

The concepts mastered in Week 9 extend directly into mobile robotics, visual odometry, and autonomous navigation.

What's Coming in Weeks 10–12

Week 10 — Mobile Kinematics

Unicycle and Ackermann models. Kinematic constraints for wheeled robots. Extends transform chain with frame W (world/map).

1

2

Week 11 — AI Line Following

Reuses the Week 8–9 idea of mapping a visual error (lane offset) to angular velocity ω for steering control.

3

Week 12 — Visual Odometry

Estimate T_{WB} from camera motion (no encoders). Combine with Week 9 mapping to build a real-time 3D map of the environment.

📌 All future weeks build directly on Week 9. Master the transform chain now and everything downstream becomes natural.

Week 9 Summary — Part 1: Concepts

1

Mapping Is the Bridge

Mapping connects the pixel/pose world of the camera to the joint/pose world of the robot. Without it, AI and kinematics remain isolated systems.

2

Hand-Eye Calibration Solves $AX = XB$

The calibration procedure finds T_{BC} — the camera's fixed pose in the robot base frame — by observing a target from multiple robot configurations.

3

$T_{CO} + T_{BC} \rightarrow T_{target}$

Combining AI pose output T_{CO} with calibration result T_{BC} (and gripper offset T_{OT}) yields T_{target} — the direct input for the Week 8 IK solver.

Week 9 Summary — Part 2: Practice

1

Error Sources Are Cumulative

AI pose errors, calibration errors, and IK errors all add up. Prioritize accuracy at the camera and calibration stage — IK cannot compensate for upstream errors.

2

Lab Exercises Build Intuition

The 8 lab exercises progressively build from single functions (`pixel_to_ray`) to a complete multi-object grasping pipeline, including error sensitivity analysis.

3

The "Bridge" Week

Week 9 is the integration week that enables the end-of-course demo: a robot that sees an object via AI and autonomously grasps it — the culmination of all previous weeks.

Discussion — Mapping & Hand-Eye Calibration

Open forum for real-world scenarios and edge cases.

“What happens if the camera mount vibrates slightly during operation? How would you detect and compensate for this?”

“If the table is slightly tilted (not perfectly horizontal), how does this affect the grasp pipeline and what correction would you apply?”

“Changing lighting conditions affect the AI 6D pose output. At what stage in the pipeline does this error appear, and how would you mitigate it?”

